

OBSERVACIONES DE LA PRÁCTICA

Jorge Claros, 202522281
Larry Thomas Sanchez, 202615358

Ambientes de pruebas

Computador 1			
Procesador	AMD	Ryzen	7
	7800X3D		
Memoria RAM (GB)	32 RAM		
Sistema Operativo	WINDOWS 11		

Tabla 1. Especificaciones del computador para ejecutar las pruebas de rendimiento.

Para realizar las siguientes pruebas, es importante que las ejecuten utilizando el archivo de datos **large**, ya que es con este conjunto donde realmente se puede observar el comportamiento y la eficiencia de los algoritmos en contextos más exigentes.

Ordenamientos Iterativos

Resultados para ordenamientos iterativos con Array List

Porcentaje de la muestra	Insertion Sort (Array List)	Selection Sort (Array List)	Shell Sort (Array List)
0.50%	0	0	0
5.00%	20	27	2
10.00%	84	107	5
20.00%	328	421	12
30.00%	747	1013	19
50.00%	2261	2748	38
80.00%	5524	7263	67
100.00%	8722	11182	82

Tabla 2. Resultados en computador 1 para ordenamientos iterativos con Array List.

Resultados para ordenamientos iterativos con Linked List

Porcentaje de la muestra	Insertion Sort (Linked List)	Selection Sort (Linked List)	Shell Sort (Linked List)
0.50%	0	0	0
5.00%	19	26	1

10.00%	77	108	4
20.00%	316	430	9
30.00%	711	981	15
50.00%	2051	2752	30
80.00%	5240	7172	57
100.00%	8196	11285	68

Tabla 3. Resultados en computador 1 para ordenamientos iterativos con Single Linked List.

Ordenamientos Recursivos

Resultados para ordenamientos recursivos con Array List

Porcentaje de la muestra	Merge Sort (Array List)	Quick Sort (Array List)
0.50%	0	0
5.00%	1	1
10.00%	3	3
20.00%	7	9
30.00%	13	17
50.00%	21	42
80.00%	37	96
100.00%	48	144

Tabla 4. Resultados en computador 1 para ordenamientos recursivos con Array List.

Resultados para ordenamientos recursivos con Linked List

Porcentaje de la muestra	Merge Sort (Linked List)	Quick Sort (Linked List)
0.50%	0	0
5.00%	3	1
10.00%	7	4
20.00%	16	9
30.00%	26	18
50.00%	60	45
80.00%	78	97
100.00%	104	143

Tabla 5. Resultados en computador 1 para ordenamientos recursivos con Single Linked List.

Una vez alla llenado l

Comparación de tiempo mejores algoritmos de ordenamiento

Complete esta sección únicamente cuando se le indique en las instrucciones. En este punto, deberá comparar el tiempo de ejecución entre el mejor algoritmo de ordenamiento recursivo y el mejor algoritmo de ordenamiento iterativo, los cuales debió haber identificado en el punto anterior.

Por favor, registre las mediciones en el espacio correspondiente de la tabla, especificando claramente:

- El nombre de cada algoritmo utilizado
- El tipo de estructura de datos empleada (*ArrayList* o *SingleLinkedList*)

Asegúrese de completar todos los campos solicitados en la tabla e incluir los nombres según corresponda, no es necesario que vuelva a correr las pruebas esta información la puede sacar de las tablas anteriores.

Computador 1

Porcentaje de la muestra	Merge Sort (ArrayList)	Shell Sort (SingleLinkedList)
0.50%	0	0
5.00%	1	1
10.00%	3	4
20.00%	7	9
30.00%	13	15
50.00%	21	30
80.00%	37	57
100.00%	48	68

Tabla 6. Resultados en computador 1 mejores algoritmos

Preguntas de análisis parte 1

1. ¿Cómo varía el comportamiento de cada algoritmo con respecto al tamaño de los datos?

Para responder, agregue líneas de tendencia en las gráficas de las pestañas 04 a 08. Observe si el crecimiento del tiempo de ejecución es lineal, cuadrático o de otro tipo, y relacione ese patrón con la complejidad teórica esperada de cada algoritmo.

RTA:

Insertion Sort y Selection Sort se vuelven muy lentos a medida que aumentas los datos. Al llegar a 10.000 registros, tardan más de 8 y 11 segundos. En cambio, Shell Sort, Merge Sort y Quick Sort manejan mucho mejor las grandes cantidades de información, ordenando los 10.000 datos en menos de un parpadeo

2. **¿Qué diferencias observas en el rendimiento de un mismo algoritmo al utilizar ArrayList frente a SingleLinkedList?**

Analice esta comparación en las pestañas 04 a 08, donde se muestran ambos casos. Justifique su respuesta con base en las líneas de tendencia.

RTA:

En Merge Sort, usar ArrayList fue más del doble de rápido que usar SingleLinkedList (48 milisegundos contra 104 milisegundos para 10.000 datos). Esto pasa porque con un arreglo la computadora puede saltar de una vez a la mitad de la lista, mientras que con la lista enlazada tiene que ir pasando elemento por elemento para dividir los datos.

3. **¿Qué algoritmo iterativo mostró el mejor comportamiento general en términos de tiempo y cantidad de datos en ambas estructuras de datos?**

Justifique su elección comparando las gráficas (04 Insertion Sort), (05 Selection Sort) y (06 Shell Sort) y apoyándose en las gráficas de las pestañas 2 y 3.

RTA:

Shell Sort. Es, por mucho, el mejor de su categoría. Mientras que los otros algoritmos iterativos tardan más de 8.000 milisegundos con 10.000 datos, Shell Sort lo logra en menos de 85 milisegundos en ambas estructuras.

4. **¿Qué algoritmo recursivo presentó el mejor desempeño general según los tiempos registrados y la forma de la línea de tendencia?**

Justifique su respuesta basándose en las pestañas 07 (Merge Sort) y 08 (Quick Sort) y apoyándose en las gráficas de las pestañas 2 y 3.

RTA:

Merge Sort. Fue el más rápido y estable de todos al procesar el 100% de los datos (48 milisegundos en ArrayList). A diferencia de Quick Sort, que se volvió un poco más lento en la prueba más grande, Merge Sort mantuvo un ritmo constante siempre.

5. **Con base en su análisis visual y los datos recolectados, ¿cuáles algoritmos seleccionan como los “mejores” en cada categoría (iterativo y recursivo)?**

Indique también la estructura de datos en la que se desempeñó mejor cada uno.

RTA:

Mejor Iterativo: Shell Sort, funcionando mejor con SingleLinkedList (68 ms).

Mejor Recursivo: Merge Sort, funcionando mejor con ArrayList (48 ms).

Preguntas de análisis parte 2

1. **¿Cuál de los dos algoritmos seleccionados como “mejores” muestra un comportamiento más eficiente a medida que crece el tamaño de la muestra?**

Para responder, agregue líneas de tendencia en la gráfica de la pestaña 09-Bests, identifique cuál algoritmo crece más lentamente y relacione ese patrón con su complejidad teórica. Use esta comparación para justificar cuál es más escalable.

RTA:

Merge Sort en ArrayList. Su tiempo de respuesta crece muy despacio a medida que aumenta el tamaño de los datos, alcanzando un tiempo máximo de solo 48 ms con 10.000 elementos frente a los 68 ms de Shell Sort. Esto se debe a su complejidad teórica de $O(N \log N)$, lo cual lo convierte en el algoritmo más escalable para grandes volúmenes de datos.

2. **¿Qué estructura de datos resultó más favorable para cada algoritmo seleccionado como mejor?**

Analice si la ventaja se mantiene constante en todos los tamaños o solo en ciertos rangos.

RTA:

Para Shell Sort, la estructura Single LinkedList resultó ligeramente mejor (68 ms frente a 82 ms en ArrayList a 10.000 datos). Para Merge Sort, la estructura ArrayList fue claramente más favorable en todos los rangos, ya que ArrayList permite el acceso directo por índice en $O(1)$, facilitando la división en mitades del algoritmo.

3. **¿Qué ventajas prácticas podría tener implementar el algoritmo ganador en aplicaciones reales donde se manejan grandes volúmenes de datos?**

RTA:

La principal ventaja de Merge Sort es que garantiza una complejidad de $O(N \log N)$ en el peor de los casos y es un algoritmo estable. En aplicaciones reales evita picos impredecibles de tiempo que sí ocurren con algoritmos de $O(N \text{ AL CUADRADO})$ cuando las bases de datos escalan a miles o millones de registros.

4. **Si se presentara una lista parcialmente ordenada, ¿cambiaría su elección de algoritmo? Explique por qué, con base en el comportamiento observado.**

RTA:

Sí. Si la lista está parcialmente ordenada, algoritmos como Insertion Sort se vuelven sumamente eficientes al aproximarse a un tiempo casi lineal $O(N)$, ya que casi no requieren desplazamientos de elementos. De igual forma, Shell Sort aprovecha este estado inicial adaptándose mejor. No obstante, si se desconoce la ordenación previa de los datos, Merge Sort sigue siendo la opción más segura por mantener su tiempo en $O(N \log N)$.

